DACON 236446 · 난독화된 한글 리뷰 복원 · 개인 모의 참여 정리

노이즈 채널에서 출발해, 문맥으로 다시 고르기까지

종료된 대회에 개인으로 모의 참여해 Public 리더보드 **0.99217**을 기록했습니다. 아래 수치는 모두 같은 검증 분할(train 의 10%, 1,126문장)의 문자 단위 F1입니다. 참고한 저장소: [SoleMin/DACON-korean_review_deobfuscator]

출발점 (노이즈 채널)
0.9834

참고 저장소 아이디어 적용 후
0.9922
KoCharELECTRA 채점기 (13b)

최종 (14)
0.9937
Public 리더보드 0.99217

문제: 같은 위치의 글자를 맞히는 복원

난독화 입력	빵위 넓꼬 쬬욯헤써 쨜 쒸얹여욥
정답	방이 넓고 조용해서 잘 쉬었어요

빨간 칸은 정답과 다른 글자. 한글 음절만 바뀌고 길이·공백·문장부호는 그대로라서, 평가는 같은 위치 글자의 일치율(F1)입니다. 이 공개본의 예시 문장은 대회 데이터가 아니라 같은 규칙으로 직접 만든 문장입니다.

전체 구조: 한 번에 풀지 않고 3단계로

① 노이즈 채널 복원
자모별 난독화 확률 × 문자 5-gram 으로 빔 서치. 어절 사전 가점.

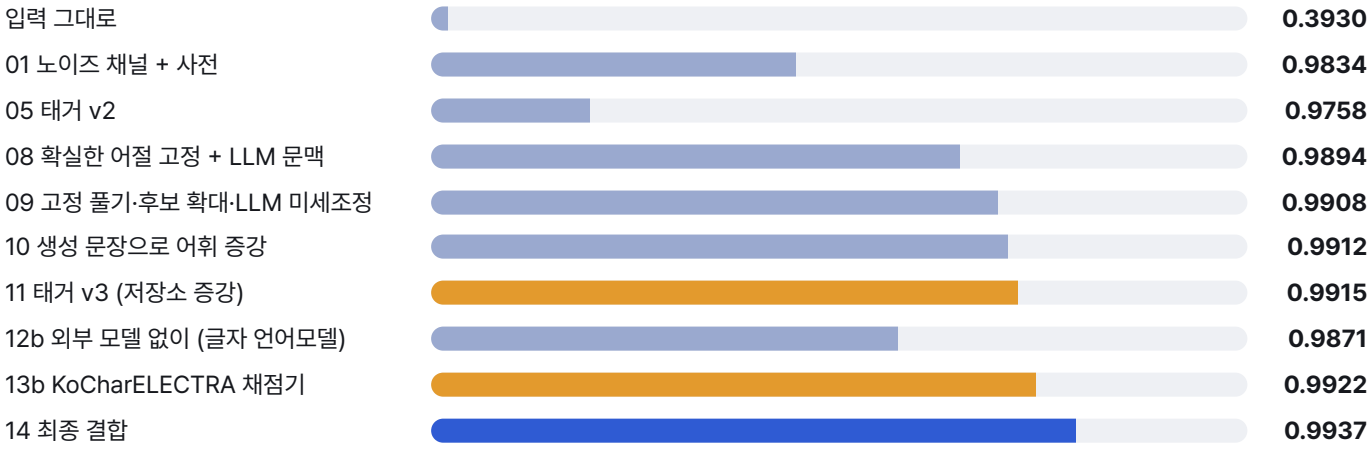
›

② 확실한 어절 고정
노이즈 채널과 태거가 같은 답을 낸 어절은 그대로 둔다 (98% 이상 정확).

›

③ 나머지를 문맥으로 재선택
후보 20개를 "난독화 유사도 + 앞뒤 문맥 점수"로 다시 고른다.

단계별 검증 F1



막대는 0.97부터 그림(차이를 보이게). 주황 막대가 참고 저장소 아이디어를 쓴 단계, 파란 막대가 최종. 05 태거는 단독 점수(이후 단계의 참조 모델로 사용), 12b 는 LLM·외부 모델 없이 낸 점수.

핵심 아이디어 ① · 처음 제시한 방식

비슷한 글자에 같은 번호를 붙여, 사전 단어와 비교한다

난독화는 글자를 아무렇게나 바꾸지 않고 **발음이 비슷한 자모끼리** 바꿉니다. 그래서 처음 세운 아이디어는 비슷한 자모에 같은 번호를 붙여 글자를 단순화하고, 단순화한 모양이 같은 사전 단어를 찾아 유사도를 비교하는 것이었습니다.

자음(초성) 번호

번호	자음
K	ㄱ ㄲ ㅋ
T	ㄷ ㄸ ㅌ
P	ㅂ ㅃ ㅍ
S	ㅅ ㅆ
J	ㅈ ㅉ ㅊ
-	ㄴ · ㄹ · ㅁ · ㅇ · ㅎ 는 각자 하나

모음(중성) 번호

번호	모음
A	ㅏ ㅑ
EO	ㅓ ㅕ ㅝ
O	ㅗ ㅛ
U	ㅜ ㅠ ㅡ
I	ㅣ ㅟ ㅢ
E	ㅐ ㅔ ㅖ ㅒ ㅚ ㅙ ㅞ

예시: 「머꺼욥」 (정답 「먹어요」)를 번호로 바꾸면

위치	입력 글자	자음 번호	모음 번호	정답 글자	자음 번호	모음 번호	자음 일치	모음 일치
1	**머** (ㅁ·ㅓ)	ㅁ	EO	**먹** (ㅁ·ㅓ·ㄱ)	ㅁ	EO	일치	일치
2	**꺼** (ㄲ·ㅓ)	K	EO	**어** (ㅇ·ㅓ)	ㅇ	EO	어긋남	일치
3	**욥** (ㅇ·ㅛ·ㅂ)	ㅇ	O	**요** (ㅇ·ㅛ)	ㅇ	O	일치	일치

모음 번호는 세 글자 모두 일치하지만, **자음 번호는 둘째 글자에서 어긋납니다.** 「먹어요」를 소리 내면 받침 ㄱ 이 다음 글자로 넘어가 「머거요」가 되는데(연음), 난독화가 이 소리를 따라 둘째 글자를 「꺼」로 적는 일이 실제로 잦습니다. 원래 초성은 ㅇ 이었는데 ㄲ 이 된 것입니다.

사전은 **번호가 정확히 같은 단어만** 꺼내 오므로, 자음 번호까지 쓰면 정답 「먹어요」는 후보에 들어오지도 못합니다.

> 이 예시는 실제 데이터에서 본 현상(연음을 따라 적기, 받침 덧붙이기)을 설명하려고 직접 만든 단어입니다.

핵심 아이디어 ② · 그래서 모음에만 적용

모음만 열쇠로, 자음·받침은 확률 점수로

위 현상이 우연인지 확인하려고 train 데이터 전체(어절 20만 6천 개)에서 각 번호가 난독화 뒤에도 그대로 남는지 셌습니다.

단순화 기준	글자 단위로 같은 번호 유지	어절 전체가 같은 번호 유지
모음 번호	**100.0%**	**100.0%**
자음(초성) 번호	95.8%	86.9%
받침(종성)	61.4% (받침을 덧붙이거나 지우는 잡음)	–

- 모음은 번호 밖으로 바뀐 경우가 **한 번도 없습니다.** 열쇠로 쓰면 정답을 절대 놓치지 않습니다.
- 자음 번호가 깨지는 경우는 거의 전부 **연음**입니다. 원래 초성 ㅇ 자리에 앞 글자 받침 소리가 넘어와 ㄹ·ㄴ·ㅅ·ㅁ·ㅎ·ㄸ 등으로 바뀝니다. 자음까지 열쇠에 넣으면 **어절 8개 중 1개(13.1%)**는 정답이 후보에서 빠집니다.
- 받침은 난독화가 덧붙이거나(「먹어요」 → 「묩」의 ㅂ) 지우는 일이 잦아 번호로 묶을 수 없습니다.

정리: 아이디어를 이렇게 나눠 적용했습니다

정보	① 사전 후보 찾기 (열쇠)	② 후보 점수 매기기
모음	**번호로 단순화해 열쇠로 사용**	확률로 사용
자음 (초성)	쓰지 않음 (연음 때문에 깨짐)	**확률로 사용** — 같은 번호 안의 변화(ㄱ→ㅋ)는 높게, 밖의 변화는 낮게 학습
받침 (종성)	쓰지 않음 (덧붙임·삭제 잡음)	**확률로 사용** — 연음은 "앞 글자 받침 + 다음 초성"을 함께 계산

즉 "비슷한 자음에 같은 번호를 붙인다"는 아이디어는 사라진 것이 아니라, **열쇠가 아닌 점수표 안으로** 옮겨졌습니다. 열쇠는 깨지지 않는 모음만으로 넓게 찾고, 고르는 단계에서 자음·받침까지 정밀하게 봅니다.

핵심 아이디어 ③ · ① 노이즈 채널 + 어절 사전

골격 단순화 → 사전 단어와 비교 → 문맥 정보 추가 → 선택

한 어절이 복원되는 과정을 「머꺼욥」(정답 「먹어요」)으로 따라가 보면 이렇습니다.

① 골격 단순화
모음 번호 열쇠
EO-EO-O
로 사전(train 어절)에서 같은 열쇠의 어절을 모두 꺼냄

›

② 사전 단어와 비교
"train 빈도 × 이 단어가 이 모양으로 난독화될 확률(자음·받침·연음 포함)"로 정규화한 사후확률

›

③ 문맥 정보 추가
후보를 문장에 끼워 앞뒤 문맥으로 자연스러움을 잼

›

④ 선택
유사도 + 문맥 점수의 합이 가장 큰 후보

- 글자 모양만 보면 받침까지 그대로 살린 「먹어욥」 같은 후보가 1위에 오기 쉽습니다. 난독화가 받침을 덧붙였기 때문입니다.
- 하지만 그런 후보는 사전에 없는 말이고 문맥 점수도 낮아서, 합계로는 정답이 올라옵니다. 실제 검증 문장에서도 글자 유사도로는 10~11위였던 정답이 문맥 점수로 1위가 된 경우가 있었습니다.

노이즈 채널과 어절 사전

모음 묶음 ㅏㅑ · ㅓㅕㅝ · ㅗㅛ · ㅜㅠㅡ · ㅣㅟㅢ · ㅐㅔㅖㅒㅚㅙㅞ
각 글자의 모음을 묶음 대표로 바꾼 "모음 골격"을 사전의 열쇠로 씁니다. 자음은 위 이유로 열쇠에서 빼고 채널 확률로만 반영합니다.

- **채널 확률**은 자모별로 "원래 자모 → 난독화 자모"가 나올 확률의 곱입니다. 연음(받침이 다음 초성 ㅇ 으로 넘어감)은 앞뒤 글자를 함께 봅니다.
- **"사전에 없음"도 하나의 후보**로 계산합니다. 처음 보는 단어일 가능성이 크면 사전 가점이 저절로 작아집니다.
- 사후확률은 글자별 가점(가중치 3.0)으로 빔 서치에 더해집니다. 언어모델은 train 정답으로 만든 문자 5-gram(Kneser-Ney). 이 단계만으로 **0.9834**.

② 확실한 어절 고정 → ③ 문맥으로 재선택

모델들이 동의한 어절은 고정하고, 나머지만 문맥으로 다시 고른다

처음에는 "긴 어절일수록 확실하다"는 기준으로 고정해 봤지만(0.9837), 검증해 보니 긴 어절이 오히려 더 자주 틀렸습니다. 대신 **노이즈 채널과 태거가 같은 답을 낸 어절**을 고정하자 98% 이상 맞았습니다(0.9894). 일치해도 그 답이 train 에 한 번도 없던 단어면 고정을 풀었습니다(이런 어절은 90%만 맞음).

처리 방식 (설명용 가상 예시)

입력	빵위 넓꼬 쬬욯헤써 쨜 쒸얹여욥
정답	방이 넓고 조용해서 잘 쉬었어요

입력 어절	두 모델의 답	처리
빵위	같음 · train 에 있는 단어	고정
넓꼬	같음 · train 에 있는 단어	고정
쬬욯헤써	같음 · train 에 있는 단어	고정
쨜	같음 · train 에 있는 단어	고정
쒸얹여욥	다름	추론 후보 20개를 문맥으로 재선택

> 표는 처리 규칙을 보이기 위한 가상 예시이며, 실제 모델 출력이 아닙니다.

- **후보**: 노이즈 채널의 답, 태거의 답, 노이즈 채널 빔 서치(빔 64)의 상위 20개.
- **최종 점수** = 난독화 유사도 + 문맥 점수(Qwen·글자 언어모델·5-gram·KoCharELECTRA).
- 고정되지 않은 어절은 긴 것부터 확정하고, 확정된 어절은 다음 어절의 문맥이 됩니다. 정방향 후 역방향으로 한 번 더 훑습니다.
- 유사도만으로 고르면 틀리는 어절을 문맥이 바로잡는 것이 이 구조의 핵심입니다. 앞 어절이 고정돼 있으면 "____ 뒤에 오기 자연스러운 말"이라는 단서가 생깁니다.

참고 저장소에서 가져온 것 ① · 태거 학습 증강

태거 v3: 부분 복원 · 관찰 형태 치환 · 이어 붙이기 · 라벨 스무딩 · EMA

참고 저장소의 증강 아이디어를 태거 학습에 넣고, 미세조정한 Qwen 이 생성한 리뷰 10만 문장에 노이즈를 입힌 샘플을 더했습니다. 태거 단독 점수가 **0.9758 → 0.9812**로 올랐고, 이 태거가 ②의 "고정 판단"과 ③의 후보에 쓰입니다.

증강 예시 (학습 문장 하나에서 만든 변형, 가상 예시)

원래 난독화 입력	빵위 넓꼬 쬬욯헤써 쨜 쒸얹여욥
정답	방이 넓고 조용해서 잘 쉬었어요
관찰된 형태로 치환	빵위 넓코 쬬융햬서 쨜 쒸얹여욥
부분 복원	빵위 넓꼬 조용해서 쨜 쉬었어요

- 주황: 그 어절이 train 의 다른 문장에서 **실제로 난독화된 모양**으로 바꿔 넣음 (가짜 노이즈가 아닌 진짜 노이즈 분포).
- 초록: 일부 어절을 미리 정답으로 되돌림. 태거가 "이미 깨끗한 단어는 건드리지 않는 법"을 배움.
- 새 샘플 이어 붙이기(test 의 긴 문장 대비), 라벨 스무딩 0.1, 가중치 EMA 0.999.

추가: 생성 리뷰로 어휘 넓히기

남은 오류의 89%가 train 에 없던 단어였습니다. train 리뷰로 미세조정한 Qwen2.5-1.5B 로 리뷰를 생성해, 노이즈를 입혀 태거 학습에 섞고 사전·5-gram 에도 새 어절을 추가했습니다(0.9908 → 0.9912). 검증용 모델은 train 90% 로만 미세조정해 검증 정답이 새지 않게 했습니다.

참고 저장소에서 가져온 것 ② · KoCharELECTRA

태거 몸통 대신 "후보 채점기"로 쓰다

참고 저장소는 KoCharELECTRA 상위 층을 태거에 얹었습니다. 여기서는 구조상 ③의 질문, 즉 **"후보 중 어느 것이 자연스러운가"**가 핵심이라, ELECTRA 의 원래 학습 과제인 **바뀐 글자 찾기(discriminator)**를 그대로 채점에 썼습니다. 가장 큰 단일 개선이었습니다(**0.9871 → 0.9922**).

계산 방법

1. 후보를 끼운 문장을 넣는다

discriminator 는 글자마다 "원래 글자가 아니라 바뀐 글자 같다"는 logit 을 냅니다. 클수록 어색한 글자입니다.

2. 후보 ± 3글자 구간을 채점한다

구간 안 글자들의 −logit 합이 그 후보의 문맥 점수입니다. 클수록 자연스럽습니다.

- 틀린 후보를 넣으면 그 글자뿐 아니라 **주변 글자까지 어색하다고 판단**하는 경우가 많았습니다. 그래서 후보 글자만이 아니라 앞뒤 3글자를 함께 채점합니다.
- logit 을 확률(sigmoid)로 바꾸면 대부분 0 근처로 몰려 차이가 사라져서, logit 을 ±10 으로 자른 값을 그대로 씁니다.
- generator(빈칸 맞히기)도 시험했지만, discriminator 와 함께 쓰면 추가 이득이 없었습니다.
- 이 채점기로 바로잡힌 예로는 받침이나 초성 하나 차이로 뜻이 달라지는 단어들이 있었습니다.

> 원본 정리 문서에는 실제 검증 문장으로 계산한 글자별 점수와 바로잡힌 문장이 있었지만, 대회 데이터라 이 공개본에서는 뺐습니다.

최종 결합 · 안 된 것 · 규칙

서로 다른 문장을 맞히는 두 방식을 합치다

LLM(Qwen) 문맥을 쓴 11b(0.9915)와 KoCharELECTRA 를 쓴 13b(0.9922)는 평균은 비슷했지만 **서로 다른 문장을 맞혔습니다**(13b 가 나은 문장 153개, 11b 가 나은 문장 136개). 그래서 문맥 점수를 모두 더했습니다.

최종 점수 = 난독화 유사도 + 1.0 × Qwen + 0.5 × 글자 언어모델 + 0.5 × 문자 5-gram + 1.0 × KoCharELECTRA → **0.9937**

- 11b 와 13b 가 둘 다 틀린 문장 중 일부(사자성어, 외래어 음식 이름 등)를 결합이 맞혔습니다.

해 보니 안 됐던 것

- 태거 점수를 노이즈 채널 빔 서치에 직접 더하기: 태거가 과신해서 하락
- 가린 어절을 맞히는 빈칸 채우기 모델을 처음부터 학습: 학습 신호 부족으로 0.9583 → 다음 글자 예측 방식(글자 언어모델)으로 바꿔 해결
- 어절 빈도 가점, "애매하면 기존 답 유지" 문턱: 효과 없음
- KoCharELECTRA generator 미세조정: discriminator 와 함께 쓰면 이득 없음

규칙과 재현

- 공개 가중치 모델만 사용: Qwen2.5-1.5B, KoCharELECTRA (둘 다 Apache-2.0). 3B·72B Qwen 은 라이선스 제약으로 제외
- 외부 데이터 없음. 생성 문장은 train 으로 미세조정한 모델이 만든 것
- 모든 가중치(문맥 점수 비율 등)는 검증 분할에서만 정하고, test 는 train 전체로 다시 학습한 모델로 예측

이 문서의 예시 문장과 단어는 모두 직접 만든 것이며 대회 데이터가 아닙니다. 수치(F1, 통계)는 대회 데이터로 계산한 결과입니다. 대회: dacon.io/competitions/official/236446